

UNIVERSIDADE TECNOLÓGICA FEDERAL DO PARANÁ

Campus Curitiba

Engenharia de Computação

EL83A – Comunicação de Dados

Codificação de Linha 8B6T em Rede Mestre-Escravo com ESP32

Slave → Master via ESP-NOW

Integrantes:

Arthur Gabriel Pellegrini Heberle

Rafael de Andrade Fernandes

Curitiba

2026

Sumário

1	Introdução	2
1.1	Contextualização	2
1.2	Objetivos	2
1.3	Organização do documento	2
2	Fundamentação Teórica	3
2.1	Codificação de linha	3
2.2	Codificação 8B6T	3
2.3	Símbolos ternários e trits	4
2.4	Cifra de Vigenère	4
2.5	Protocolo ESP-NOW	4
3	Arquitetura do Sistema	5
3.1	Visão geral	5
3.2	Topologia e papéis dos nós	5
3.3	Fluxo de comunicação	5
3.4	Componentes de hardware	6
3.5	Componentes de software	6
4	Implementação	6
4.1	Visão geral do módulo <code>encode.py</code>	6
4.2	Cifra de Vigenère: <code>encodeMessage</code> e <code>decode_message</code>	6
4.3	Conversão texto-binário: <code>textToBinary</code>	7
4.4	Codificação 8B6T: <code>binaryToTrits</code>	7
4.5	Decodificação 8B6T: <code>tritsToBinary</code>	8
4.6	Tabela 8B6T	8
4.7	Firmware dos ESP32	8
4.8	Interface gráfica	8
5	Resultados e Discussão	8
6	Conclusão	8

1 Introdução

1.1 Contextualização

Este relatório descreve o projeto de implementação de codificação de linha desenvolvido na disciplina de Comunicação de Dados (EL83A). O trabalho implementa o algoritmo 8B6T sobre uma rede de três microcontroladores ESP32, organizados em topologia mestre-escravo, com interface gráfica em Python rodando em computadores conectados via porta serial a cada ESP32.

O sistema integra três etapas distintas: criptografia da mensagem por meio da cifra de Vigenère, conversão do texto cifrado para uma representação binária e, por fim, mapeamento de cada byte em uma sequência de seis símbolos ternários (trits) conforme a tabela 8B6T. A forma de onda ternária resultante é exibida em ambas as interfaces gráficas.

1.2 Objetivos

Objetivo geral: demonstrar, de ponta a ponta, a transmissão de uma mensagem entre um nó escravo e o nó mestre, aplicando criptografia simétrica de chave pré-compartilhada e codificação de linha 8B6T, com visualização da forma de onda ternária em ambos os lados da comunicação.

Objetivos específicos:

- Implementar a cifra de Vigenère com chave simétrica pré-compartilhada para proteção da mensagem (T4).
- Converter o texto cifrado em binário utilizando a codificação ASCII estendido, com suporte a caracteres especiais e acentuados (T5).
- Aplicar o algoritmo 8B6T para mapear cada byte em seis trits $\{-1, 0, +1\}$ (T6).
- Exibir a interface gráfica com mensagem original, mensagem criptografada, representação binária, trits e forma de onda ternária (T1, T2).
- Transmitir o payload de trits via ESP-NOW entre os ESP32, com comunicação serial entre cada ESP32 e seu computador (T3, T7).
- Realizar o processo inverso completo no lado do mestre: recuperar os trits, reconstruir o binário, decodificar o texto e decifrar a mensagem original (T8).

1.3 Organização do documento

A Seção 2 apresenta a fundamentação teórica dos conceitos utilizados. A Seção 3 descreve a arquitetura do sistema, incluindo hardware, software e topologia de rede. A

Seção 4 detalha a implementação de cada etapa do pipeline. A Seção 5 apresenta os resultados obtidos. A Seção 6 conclui o trabalho.

2 Fundamentação Teórica

2.1 Codificação de linha

Codificação de linha é o processo de converter uma sequência de bits em um sinal elétrico adequado para transmissão em um meio físico. Um bom esquema de codificação de linha busca, entre outras propriedades: ausência de componente DC (balanceamento), densidade de transições suficiente para recuperação de relógio no receptor e eficiência de banda.

Esquemas ternários, como o 8B6T, utilizam três níveis de amplitude de sinal. Isso os diferencia dos esquemas binários, que utilizam apenas dois níveis. Com três níveis, é possível representar mais informação por símbolo transmitido, o que contribui para a eficiência espectral.

No contexto deste projeto, o meio físico de transmissão entre os ESP32 é o rádio WiFi (protocolo ESP-NOW), de modo que os trits não se traduzem diretamente em níveis de tensão num condutor. O algoritmo 8B6T é aplicado como transformação de dados: cada byte é mapeado em seis trits, que são serializados e transmitidos como payload digital, e a forma de onda ternária é reconstruída e exibida nas interfaces gráficas de ambos os lados.

2.2 Codificação 8B6T

O algoritmo 8B6T mapeia cada octeto (8 bits) em um grupo de 6 símbolos ternários, onde cada símbolo pertence ao conjunto $\{-1, 0, +1\}$, também representados por $\{-, 0, +\}$. A relação entre o espaço de entrada e o espaço de saída é:

$$2^8 = 256 \text{ possíveis bytes} \quad \longrightarrow \quad 3^6 = 729 \text{ possíveis sequências de 6 trits}$$

A folga de $729 - 256 = 473$ combinações não utilizadas permite selecionar um subconjunto de 256 padrões com boas propriedades de balanceamento de componente DC e de densidade de transições. O 8B6T é historicamente associado ao padrão Ethernet 100BASE-T4 (IEEE 802.3), no qual seis símbolos ternários são transmitidos sobre pares trançados. O mapeamento completo é definido por uma tabela estática, utilizada tanto na codificação quanto na decodificação.

A eficiência de banda do 8B6T pode ser calculada como a razão entre os bits de informação transmitidos e os símbolos necessários:

$$\eta = \frac{8 \text{ bits}}{6 \text{ símbolos}} \approx 1,33 \text{ bits/símbolo}$$

2.3 Símbolos ternários e trits

Um *trit* é um dígito ternário, capaz de assumir exatamente três valores. Neste projeto, os trits assumem os valores inteiros -1 , 0 e $+1$, associados a níveis de amplitude da forma de onda:

trit $+1 \rightarrow$ amplitude positiva, trit $0 \rightarrow$ amplitude neutra, trit $-1 \rightarrow$ amplitude negativa

Na interface gráfica, a sequência de trits é exibida como uma forma de onda em degraus, onde cada trit corresponde a um intervalo de tempo com amplitude constante em um dos três níveis.

2.4 Cifra de Vigenère

A cifra de Vigenère é uma cifra de substituição polialfa bética que aplica deslocamentos distintos a cada caractere da mensagem, determinados por uma chave repetida ciclicamente. Nesta implementação, os caracteres são tratados pelos seus valores numéricos e a operação ocorre no intervalo de um byte (0 a 255):

$$c_i = (m_i + k_{i \bmod |k|}) \bmod 256$$

$$m_i = (c_i - k_{i \bmod |k|} + 256) \bmod 256$$

onde m_i é o valor numérico do i -ésimo caractere da mensagem original, k_j é o valor numérico do j -ésimo caractere da chave, c_i é o caractere cifrado resultante e $|k|$ é o comprimento da chave. O termo $+256$ na decodificação evita resultados negativos antes da aplicação do módulo.

A chave é pré-compartilhada entre os dois lados da comunicação e não é transmitida junto com a mensagem. A segurança do esquema depende inteiramente do sigilo da chave, pois o algoritmo em si é conhecido.

2.5 Protocolo ESP-NOW

O ESP-NOW é um protocolo de comunicação sem fio desenvolvido pela Espressif para microcontroladores ESP32 e ESP8266. Ele opera na camada de enlace, abaixo do IP e do TCP/IP, sem necessidade de roteador ou ponto de acesso. Cada dispositivo é identificado pelo seu endereço MAC. Um nó envia um payload de até 250 bytes diretamente ao endereço MAC do destinatário, sem estabelecimento prévio de conexão.

No contexto deste projeto, o payload ESP-NOW contém a sequência de trits serializada, originada no ESP32 escravo e destinada ao ESP32 mestre.

3 Arquitetura do Sistema

3.1 Visão geral

O sistema é composto por dois computadores com interface gráfica e três ESP32. Cada computador conecta-se ao seu ESP32 via porta serial (UART). Os ESP32 comunicam-se entre si via ESP-NOW. A Figura 1 ilustra a arquitetura completa.

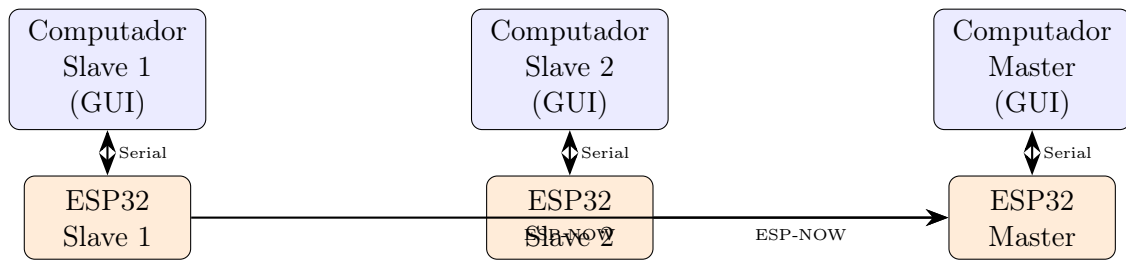


Figura 1: Arquitetura do sistema: dois escravos e um mestre, cada um com GUI em computador dedicado, comunicando-se via ESP-NOW entre os ESP32.

3.2 Topologia e papéis dos nós

O **nó escravo** (Slave 1 ou Slave 2) é o lado emissor. É nele que ocorre todo o pipeline de codificação: o usuário digita a mensagem na GUI, que aplica a cifra de Vigenère, converte para binário, aplica o 8B6T e exibe a forma de onda ternária. Os trits são então serializados e enviados ao ESP32 escravo via serial, que os transmite ao ESP32 mestre via ESP-NOW.

O **nó mestre** (Master) é o lado receptor. O ESP32 mestre recebe o payload via ESP-NOW e o repassa ao computador master via serial. A GUI master recebe os dados, desserializa os trits, exibe a forma de onda ternária e executa o pipeline inverso: decodificação 8B6T, reconstrução do binário, conversão para texto e decifragem de Vigenère.

O master não interpreta nem recodifica o conteúdo: ele atua como receptor final. A inteligência de decodificação reside inteiramente na GUI do computador master.

3.3 Fluxo de comunicação

A Figura 2 apresenta o fluxo completo da mensagem, da entrada no escravo até a exibição no mestre.

3.4 Componentes de hardware

- $3 \times$ ESP32 (modelo a confirmar) — um mestre e dois escravos.
- $3 \times$ computadores com porta USB para conexão serial.
- Cabos USB para conexão ESP32-computador.

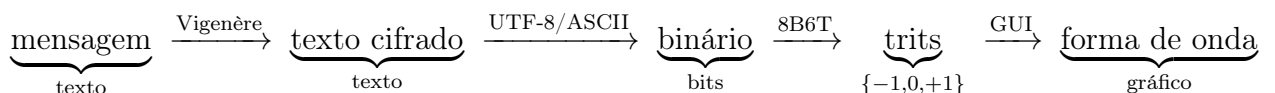
3.5 Componentes de software

- Interface gráfica em Python com PySide6, pyserial e pyqtgraph.
- Módulo de codificação `encode.py` (cifra de Vigenère, 8B6T, conversão binária).
- Firmware ESP32 para recepção serial, transmissão ESP-NOW (escravo) e recepção ESP-NOW com repasse serial (mestre).

4 Implementação

4.1 Visão geral do módulo `encode.py`

O arquivo `encode.py` concentra as funções de criptografia e codificação. O pipeline que ele implementa é:



4.2 Cifra de Vigenère: `encodeMessage` e `decode_message`

A função `encodeMessage` implementa a cifragem. Para cada caractere da mensagem, ela obtém o valor numérico do caractere e do caractere correspondente da chave (com repetição cíclica) e realiza a soma modular em 256:

```
1 def encodeMessage(message: str, encode_key: str) -> str:
2     if not encode_key:
3         raise ValueError("Encode key cannot be empty.")
4     output = ""
5     for i, char in enumerate(message):
6         message_value = ord(char)
7         key_value = ord(encode_key[i % len(encode_key)])
8         encoded_value = (message_value + key_value) % 256
9         output += chr(encoded_value)
10    return output
```

A função `decode_message` é a inversa: subtrai o valor da chave ao invés de somar, adicionando 256 antes do módulo para evitar resultados negativos:

```
1 def decode_message(encoded_message: str, encode_key: str) -> str:
2     if not encode_key:
3         raise ValueError("Encode key cannot be empty.")
4     output = ""
5     for i, char in enumerate(encoded_message):
6         encoded_value = ord(char)
7         key_value = ord(encode_key[i % len(encode_key)])
8         decoded_value = (encoded_value - key_value + 256) % 256
9         output += chr(decoded_value)
10    return output
```

A chave é pré-compartilhada entre o escravo e o mestre e não transita pela rede.

4.3 Conversão texto-binário: `textToBinary`

A função `textToBinary` converte cada byte da mensagem cifrada em sua representação binária de 8 bits:

```
1 def textToBinary(message: str) -> str:
2     return "".join(format(byte, "08b") for byte in message.encode("utf-8"))
```

4.4 Codificação 8B6T: `binaryToTrits`

A função `binaryToTrits` percorre a string binária em blocos de 8 bits e consulta a tabela 8B6T para obter a sequência de 6 trits correspondente a cada byte:

```
1 def binaryToTrits(binary_message: str, table_path: str = "") ->
2     list[int]:
3     # Valida que a mensagem contem apenas 0 e 1
4     # Valida que o comprimento e multiplo de 8
5     # Le a tabela do arquivo externo (CSV ou TXT)
6     trits = []
7     for i in range(0, len(binary_message), 8):
8         byte = binary_message[i : i + 8]
9         trits.extend(table[byte])
10    return trits
```

A tabela é carregada de um arquivo externo (`tabela_8b6t.csv` ou `tabela_8b6t.txt`). No formato texto, cada linha contém um byte em binário seguido de seis símbolos -, 0 ou +, que são convertidos para -1, 0 e +1.

4.5 Decodificação 8B6T: tritsToBinary

A função `tritsToBinary` deve realizar o caminho inverso: receber uma lista de trits e reconstruir a string binária original, consultando a tabela de forma invertida (de trits para byte). A implementação atual contém um erro: a variável `trits` é sobrescrita dentro do laço, perdendo os dados de entrada, e as fatias usadas para consultar a tabela são incorretas (usam `i:i+8` ao invés de `i:i+6`). Esse ponto deve ser corrigido antes da apresentação para atender ao requisito T8.

4.6 Tabela 8B6T

A tabela de mapeamento 8B6T utilizada no projeto contém 256 entradas, uma para cada valor de byte possível (de 00000000 a 11111111). Cada entrada associa o byte a uma sequência de seis símbolos ternários. A tabela é estática e idêntica em ambos os lados da comunicação.

4.7 Firmware dos ESP32

4.8 Interface gráfica

A interface gráfica é implementada em Python com PySide6 e exibe, em ambos os computadores (escravo e mestre), a forma de onda ternária correspondente aos trits da mensagem codificada. O gráfico é gerado com `pyqtgraph` e representa cada trit como um nível constante de amplitude ao longo de um intervalo de tempo unitário.

5 Resultados e Discussão

6 Conclusão

Referências

- [1] FOROUZAN, B. A. *Comunicação de Dados e Redes de Computadores*. 4. ed. Porto Alegre: AMGH/McGraw-Hill, 2008.
- [2] IEEE. *IEEE Std 802.3-2018: Ethernet*. Nova York: IEEE, 2018. (Cláusula relativa ao 100BASE-T4 e codificação 8B6T.)
- [3] ESPRESSIF SYSTEMS. *ESP-NOW User Guide*. Disponível em: https://docs.espressif.com/projects/esp-idf/en/latest/esp32/api-reference/network/esp_now.html. Acesso em: 2026.

- [4] KAHN, D. *The Codebreakers: The Comprehensive History of Secret Communication*.
Nova York: Scribner, 1996.



Figura 2: Fluxo completo de comunicação, do lado escravo ao lado mestre.